

Laporan Praktikum Kontrol Cerdas

(Praktikum 13 MediaPipe Hand)

Nama : Rifaldi Ahnaf Fahreza
NIM : 234308082
Kelas : TKA – 6C
Github : <https://github.com/rifaldiAhnafFahreza>

Abstrak

Praktikum ini bertujuan untuk mengimplementasikan sistem deteksi dan analisis tangan secara real-time menggunakan library OpenCV dan MediaPipe pada bahasa pemrograman Python. Modul MediaPipe Hands digunakan untuk mendeteksi keberadaan tangan, menampilkan 21 titik landmark beserta garis penghubungnya, serta mengklasifikasikan tangan kanan dan kiri berdasarkan fitur handedness. Sistem bekerja dengan mengambil citra dari webcam, mengonversi format warna dari BGR ke RGB, kemudian memprosesnya untuk menghasilkan data koordinat landmark dalam bentuk nilai ter-normalisasi. Selain itu, dilakukan analisis sederhana untuk membedakan orientasi tangan (telapak atau punggung tangan) dengan membandingkan posisi koordinat ujung jempol dan ujung kelingking. Hasil praktikum menunjukkan bahwa OpenCV dan MediaPipe dapat diimplementasikan secara efektif untuk membangun sistem interaksi berbasis gesture tanpa sensor tambahan. Sistem yang dikembangkan mampu menjadi dasar dalam pengembangan aplikasi Human-Computer Interaction (HCI) berbasis pengenalan gerakan tangan yang lebih kompleks.

Kata kunci: *Computer Vision, MediaPipe Hands, OpenCV, Hand Landmark, Gesture Recognition.*

1. Pendahuluan

Berbagai Perkembangan teknologi *computer vision* telah mendorong terciptanya sistem interaksi manusia dan komputer (*Human-Computer Interaction/HCI*) yang lebih alami tanpa menggunakan perangkat input konvensional seperti mouse dan keyboard (Lugaresi dkk., t.t.). Salah satu pendekatan yang berkembang pesat adalah pengenalan gestur tangan (*hand gesture recognition*), karena tangan merupakan alat komunikasi non-verbal yang intuitif dan mudah digunakan dalam berbagai aplikasi interaktif (Zhang dkk., 2020).

Salah satu framework yang banyak digunakan dalam pengembangan sistem berbasis visi komputer adalah MediaPipe, sebuah framework sumber terbuka yang dikembangkan oleh Google untuk membangun *perception pipeline* secara efisien dan real-time (Lugaresi dkk., t.t.). Framework ini mendukung berbagai modul, termasuk deteksi wajah, pose tubuh, dan pelacakan tangan.

Modul MediaPipe Hands secara khusus dirancang untuk melakukan pelacakan tangan (*hand tracking*) secara real-time hanya dengan menggunakan kamera RGB tanpa sensor tambahan (Zhang dkk., 2020). Sistem ini bekerja dengan dua tahap utama, yaitu deteksi telapak tangan (*palm detection*) dan estimasi 21 titik koordinat (*hand landmarks*) tiga dimensi yang merepresentasikan pergelangan tangan, ruas jari, dan ujung jari (Zhang dkk., 2020). Representasi landmark ini memungkinkan analisis posisi dan orientasi tangan secara akurat.

Selain mendeteksi posisi tangan, MediaPipe Hands juga menyediakan fitur klasifikasi untuk membedakan tangan kanan (*right hand*) dan tangan kiri (*left hand*) berdasarkan model pembelajaran mesin yang telah dilatih sebelumnya (Zhang dkk., 2020). Informasi koordinat landmark yang dihasilkan juga dapat digunakan untuk menganalisis orientasi tangan, sehingga memungkinkan pembedaan antara kondisi telapak tangan menghadap kamera dan punggung tangan menghadap kamera melalui analisis susunan titik-titik jari (Diógenes Do Rego & De Sousa, 2021).

Visualisasi garis penghubung antar landmark (*hand connections*) yang ditampilkan secara real-time membantu dalam memahami struktur anatomi tangan secara digital serta mempermudah proses analisis gerakan dan pengembangan sistem berbasis gestur (Zhang dkk., 2020). Pendekatan ini telah banyak digunakan dalam penelitian pengembangan sistem kontrol virtual, bahasa isyarat, dan antarmuka tanpa sentuhan dengan tingkat akurasi yang baik (Diógenes Do Rego & De Sousa, 2021).

Pada praktikum ini, dilakukan percobaan menggunakan MediaPipe Hands untuk mengidentifikasi tangan kanan dan kiri, membedakan telapak tangan dan punggung tangan berdasarkan orientasi landmark, serta menampilkan garis penghubung antar titik tangan secara real-time.

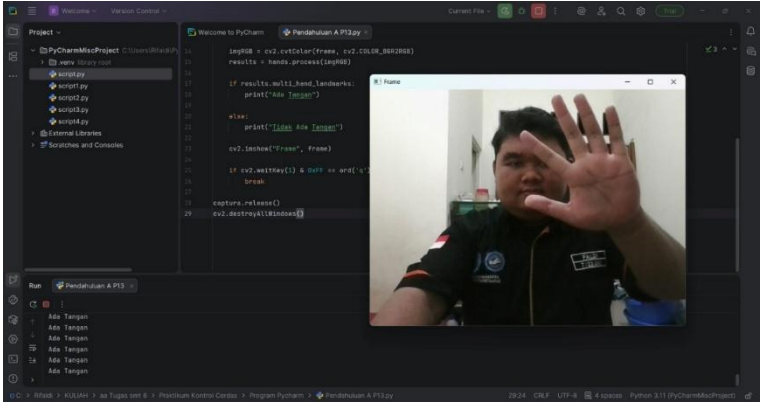
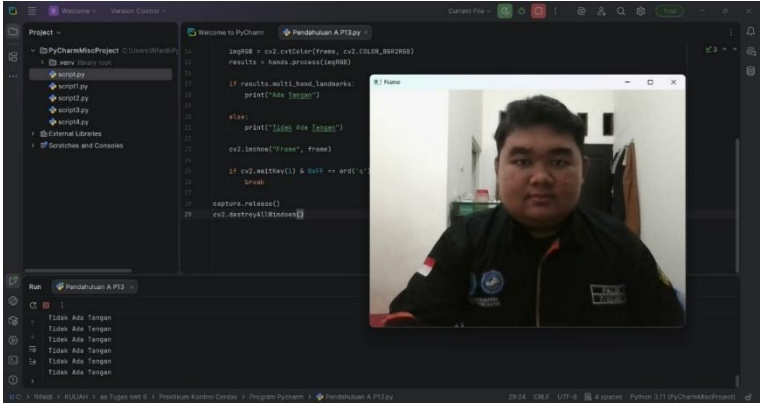
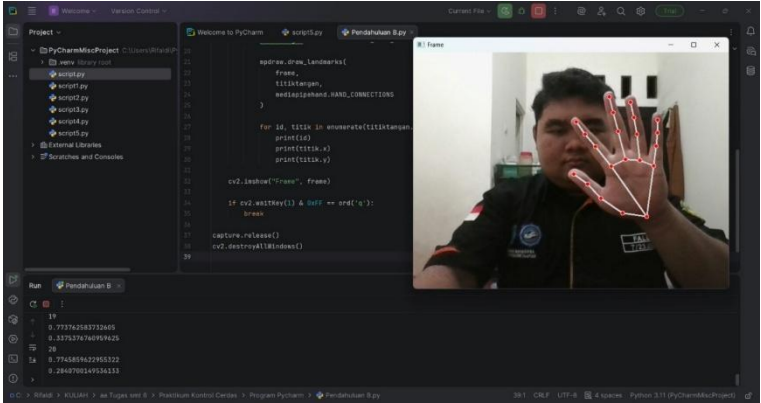
2. Tujuan

- 2.1 Memahami konsep dasar computer vision menggunakan Python.
- 2.2 Memahami konsep mediapipe
- 2.3 Menggunakan MediaPipe untuk mendeteksi dan melacak tangan secara real-time.
- 2.4 Mempermudah identifikasi orientasi tangan (telapak atau punggung) secara otomatis tanpa sensor tambahan
- 2.5 Mengidentifikasi dan mengklasifikasikan tangan kanan dan tangan kiri.

3. Manfaat

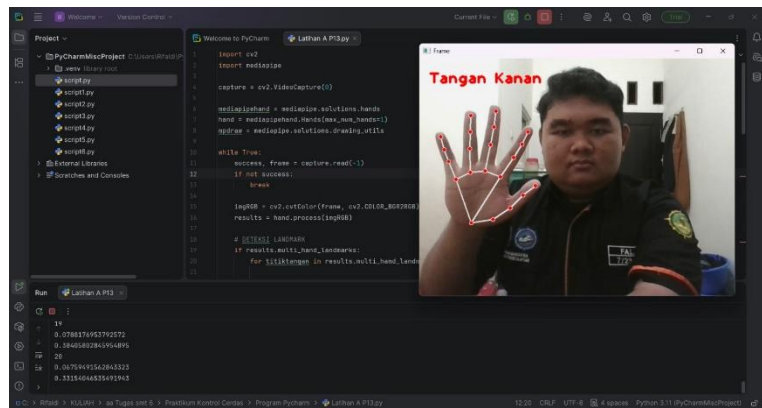
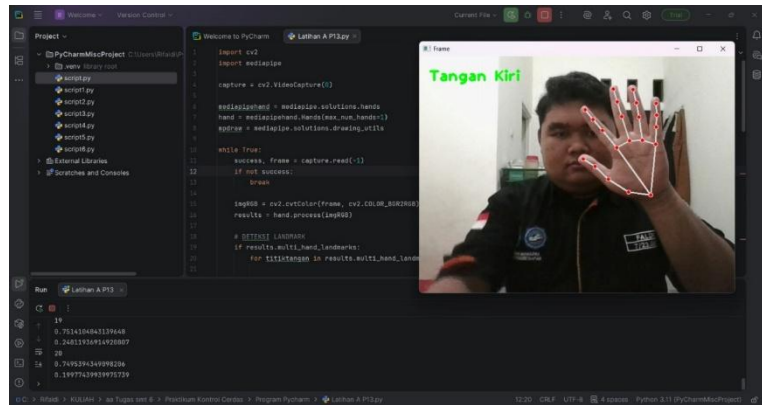
- 3.1 Meningkatkan pemahaman tentang pemrosesan citra digital dan computer vision.
- 3.2 Meningkatkan pemahaman mediapipe
- 3.3 Melatih kemampuan pemrograman Python dalam pengolahan data visual.
- 3.4 Memberikan dasar pengembangan sistem berbasis gesture recognition

4. Output hasil running program

No.	Tugas	Hasil dan Dokumentasi
1	Pendahuluan A	 
2	Pendahuluan B	

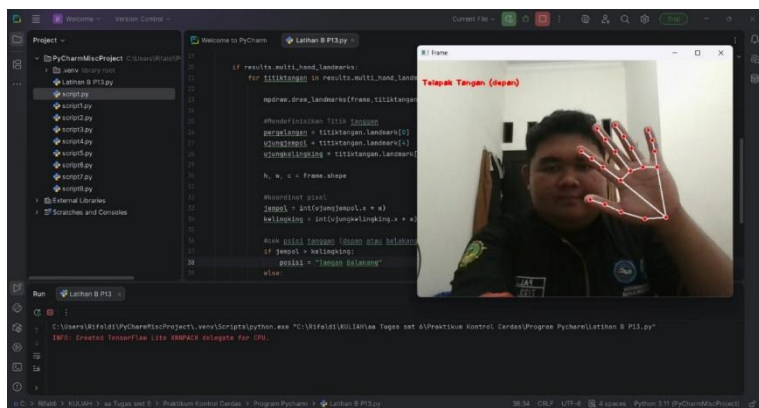
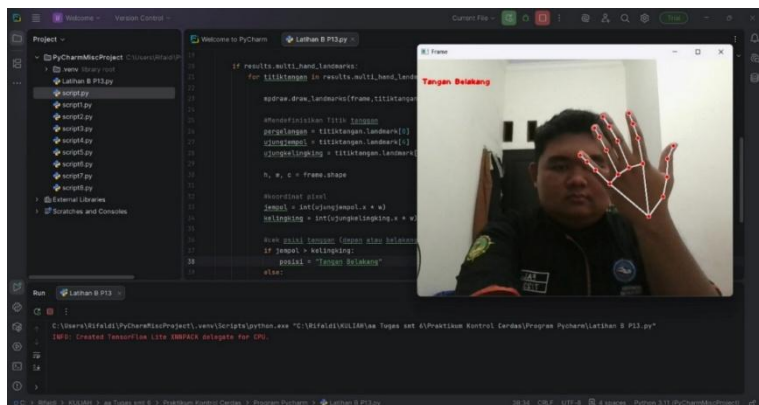
3

Latihan A



4

Latihan B



5. Analisis program

5.1 Pendahuluan A

```
1 import cv2 #import module opencv
2 import mediapipe as mp
3
4 capture = cv2.VideoCapture(0) #video capture pada device kamera nomer(0)
5
6 mpHands = mp.solutions.hands #inisialisasi deteksi tangan
7 hands = mpHands.Hands() #variable tangan untuk menyimpan konfigurasi deteksi tangan
8
9 while True:
10     success, frame = capture.read() #menyimpan citra tangkapan kamera ke frame
11     if not(success):
12         break
13
14     imgRGB = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB) #merubah warna img ke RGB
15     results = hands.process(imgRGB) #melakukan pemrosesan dari citra ke imgRGB
16
17     if results.multi_hand_landmarks: #ketika tangan terdeteksi maka akan muncul "ada tangan" pada terminal
18         print("Ada Tangan")
19
20     else:
21         print("Tidak Ada Tangan") #ketika tangan tidak terdeteksi maka akan muncul "tidak ada tangan" pada terminal
22
23     cv2.imshow("Frame", frame)
24
25     if cv2.waitKey(1) & 0xFF == ord('q'):
26         break
27
28 capture.release() #menutup webcam dengan menekan tombol q
29 cv2.destroyAllWindows()
```

Pada tahap awal dilakukan proses import library OpenCV dan MediaPipe. Library OpenCV digunakan untuk pengolahan citra digital dan pengambilan data dari kamera, sedangkan MediaPipe digunakan sebagai modul pendeteksi tangan. Sumber citra diperoleh dari webcam dengan indeks perangkat 0 melalui perintah `cv2.VideoCapture(0)`, yang kemudian disimpan dalam variabel `capture`. Perintah ini berfungsi untuk mengaktifkan kamera sebagai media input sistem.

Selanjutnya dilakukan inisialisasi modul `mp.solutions.hands` yang disimpan dalam variabel `mpHands`. Objek `Hands()` kemudian dibuat dan disimpan pada variabel `hands`. Objek ini berfungsi sebagai model pendeteksi tangan yang akan memproses citra dan mengidentifikasi keberadaan landmark tangan. Proses utama program berjalan di dalam perulangan `while True`, yang memungkinkan sistem membaca frame video secara terus-menerus (real-time). Setiap iterasi akan mengambil satu frame dari kamera menggunakan `capture.read()`. Jika proses pembacaan frame gagal, maka perulangan akan dihentikan untuk mencegah kesalahan sistem. Frame yang diperoleh dari kamera masih dalam format warna BGR (standar OpenCV). Oleh karena itu, dilakukan konversi warna ke format RGB menggunakan fungsi `cv2.cvtColor()`, karena MediaPipe memerlukan input dalam format RGB untuk melakukan proses deteksi.

Citra RGB yang telah dikonversi kemudian diproses menggunakan `hands.process(imgRGB)`. Hasil pemrosesan disimpan dalam variabel `results`. Apabila sistem mendeteksi keberadaan tangan, maka atribut `results.multi_hand_landmarks` akan bernilai tidak kosong. Dalam kondisi tersebut, sistem menampilkan teks “Ada Tangan” pada terminal. Sebaliknya, apabila tidak terdapat tangan yang terdeteksi, maka sistem akan menampilkan teks “Tidak Ada Tangan”.

Hasil tampilan kamera ditampilkan secara real-time menggunakan fungsi `cv2.imshow()`. Program akan terus berjalan hingga pengguna menekan tombol ‘q’, yang berfungsi sebagai perintah untuk menghentikan sistem. Setelah proses dihentikan, kamera dilepaskan menggunakan `release()` dan seluruh jendela ditutup menggunakan `destroyAllWindows()` untuk memastikan tidak ada sumber daya yang masih digunakan.

5.2 Pendahuluan B

```
1  import cv2 # import module opencv
2  import mediapipe
3
4  capture = cv2.VideoCapture(0)
5
6  mediapipehand = mediapipe.solutions.hands
7  hand = mediapipehand.Hands(max_num_hands=1)
8  mpdraw = mediapipe.solutions.drawing_utils
9
10 while True:
11     success, frame = capture.read()
12     if not success:
13         break
14
15     imgRGB = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
16     results = hand.process(imgRGB)
17
18     if results.multi_hand_landmarks:
19         for titiktangan in results.multi_hand_landmarks:
20
21             mpdraw.draw_landmarks(
22                 frame,
23                 titiktangan,
24                 mediapipehand.HAND_CONNECTIONS
25             )
26         for id, titik in enumerate(titiktangan.landmark):
27             print(id)
28             print(titik.x)
29             print(titik.y)
```

```

30
31     cv2.imshow("Frame", frame)
32
33     if cv2.waitKey(1) & 0xFF == ord('q'):
34         break
35
36     capture.release()
37     cv2.destroyAllWindows()

```

Pada tahap awal dilakukan proses import library OpenCV dan MediaPipe serta inisialisasi kamera menggunakan *cv2.VideoCapture(0)*. Proses ini sama dengan program sebelumnya, yaitu menggunakan webcam sebagai sumber input citra secara real-time.

Selanjutnya dilakukan inisialisasi modul *mediapipe.solutions.hands* yang disimpan dalam variabel *mediapipehand*. Berbeda dengan program sebelumnya, pada program ini ditambahkan parameter *max_num_hands=1* pada saat pembuatan objek *Hands()*. Parameter tersebut berfungsi untuk membatasi jumlah tangan yang dapat dideteksi sistem, yaitu maksimal satu tangan. Pembatasan ini bertujuan untuk meningkatkan fokus deteksi serta mengurangi beban pemrosesan apabila sistem hanya memerlukan satu objek tangan.

Variabel *mpdraw* diinisialisasi dari *mediapipe.solutions.drawing_utils*, yang berfungsi untuk menggambar landmark dan koneksi antar titik tangan pada frame video. Proses pembacaan frame, pengecekan keberhasilan pembacaan, serta konversi warna dari BGR ke RGB sama dengan program sebelumnya. Konversi ini tetap diperlukan karena MediaPipe memproses citra dalam format RGB.

Hasil pemrosesan citra disimpan dalam variabel *results*. Apabila sistem mendeteksi tangan, maka *results.multi_hand_landmarks* akan berisi data landmark tangan. Program kemudian melakukan perulangan untuk setiap tangan yang terdeteksi (meskipun telah dibatasi satu tangan).

Fungsi *mpdraw.draw_landmarks()* digunakan untuk menggambar titik-titik landmark serta garis penghubung antar titik pada tangan. Parameter *mediapipehand.HAND_CONNECTIONS* berfungsi untuk menentukan pola koneksi antar landmark sehingga membentuk struktur tangan yang utuh pada tampilan layar.

Selanjutnya dilakukan perulangan menggunakan `enumerate()` terhadap `titiktangan.landmark`. Setiap landmark memiliki:

`id` → nomor indeks titik (0–20),

`titik.x` → koordinat horizontal (nilai ter-normalisasi 0–1),

`titik.y` → koordinat vertikal (nilai ter-normalisasi 0–1).

Nilai koordinat tersebut dicetak pada terminal. Data ini dapat dimanfaatkan untuk pengembangan lebih lanjut, seperti klasifikasi gesture, perhitungan jarak antar jari, maupun analisis posisi tangan.

5.3 Latihan A

```
1  import cv2
2  import mediapipe
3
4  capture = cv2.VideoCapture(0)
5
6  mediapipehand = mediapipe.solutions.hands
7  hand = mediapipehand.Hands(max_num_hands=1)
8  mpdraw = mediapipe.solutions.drawing_utils
9
10 while True:
11     success, frame = capture.read()
12     if not success:
13         break
14
15     imgRGB = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
16     results = hand.process(imgRGB)
17
18     # ===== DETEKSI LANDMARK =====
19     if results.multi_hand_landmarks:
20         for titiktangan in results.multi_hand_landmarks:
21             mpdraw.draw_landmarks(
22                 frame,
23                 titiktangan,
24                 mediapipehand.HAND_CONNECTIONS
25             )
26
```

```
28     for id, titik in enumerate(titiktangan.landmark):
29         print(id)
30         print(titik.x)
31         print(titik.y)
32
33     # ===== KLASIFIKASI TANGAN KANAN / KIRI =====
34     if results.multi_handedness:
35         for idx, hand_handedness in enumerate(results.multi_handedness):
36
37             label = hand_handedness.classification[0].label
38
39             if label == "Right":
40                 cv2.putText(frame, "Tangan Kiri",
41                             (20, 50),
42                             cv2.FONT_HERSHEY_PLAIN,
43                             2, (0, 255, 0), 3)
44
45             elif label == "Left":
46                 cv2.putText(frame, "Tangan Kanan",
47                             (20, 50),
48                             cv2.FONT_HERSHEY_PLAIN,
49                             2, (0, 0, 255), 3)
50
51             cv2.imshow("Frame", frame)
52
53             if cv2.waitKey(1) & 0xFF == ord('q'):
54                 break
55
56 capture.release()
57 cv2.destroyAllWindows()
```

Program ini merupakan pengembangan dari sistem deteksi tangan sebelumnya karena tidak hanya mendeteksi keberadaan tangan dan menampilkan landmark, tetapi juga melakukan klasifikasi tangan kanan dan kiri secara real-time. Pada tahap awal dilakukan proses import library OpenCV dan MediaPipe serta inisialisasi kamera menggunakan `cv2.VideoCapture(0)` sebagai sumber input citra. Proses ini sama dengan program sebelumnya, yaitu memanfaatkan webcam untuk memperoleh frame video secara berkelanjutan.

Selanjutnya dilakukan inisialisasi modul `mediapipe.solutions.hands` dengan parameter `max_num_hands=1`, yang bertujuan untuk membatasi sistem hanya mendeteksi satu tangan. Pembatasan ini membantu meningkatkan efisiensi pemrosesan serta memfokuskan sistem pada satu objek utama. Variabel tambahan dari `mediapipe.solutions.drawing_utils` digunakan untuk menggambar landmark tangan pada frame video.

Proses pembacaan frame, pengecekan keberhasilan pengambilan citra, serta konversi warna dari format BGR ke RGB sama dengan program sebelumnya. Konversi warna diperlukan karena MediaPipe menggunakan format RGB sebagai standar input pemrosesan model deteksi.

Pada bagian deteksi landmark, sistem memeriksa apakah terdapat tangan yang terdeteksi melalui atribut `results.multi_hand_landmarks`. Jika tangan terdeteksi, sistem akan menggambar titik-titik landmark beserta garis penghubung antar titik sehingga membentuk struktur tangan secara visual pada layar. Selain itu, dilakukan proses enumerasi terhadap setiap landmark untuk memperoleh nomor indeks titik serta koordinat posisi dalam bentuk nilai ter-normalisasi (rentang 0 hingga 1). Data koordinat ini ditampilkan pada terminal dan dapat dimanfaatkan untuk pengembangan sistem lanjutan seperti analisis posisi jari atau pengenalan gesture.

Pengembangan utama pada program ini terletak pada fitur klasifikasi tangan kanan dan kiri. Sistem memanfaatkan atribut `results.multi_handedness` untuk membaca hasil klasifikasi dari model MediaPipe. Label yang dihasilkan berupa “*Right*” atau “*Left*” yang menunjukkan jenis tangan yang terdeteksi. Informasi tersebut kemudian ditampilkan pada layar menggunakan fungsi `cv2.putText()` sehingga pengguna dapat mengetahui apakah tangan yang terdeteksi merupakan tangan kanan atau tangan kiri. Pada implementasi ini, teks

yang ditampilkan terlihat terbalik terhadap label asli, yang umumnya disebabkan oleh efek pencerminan (*mirror*) dari kamera webcam.

5.4 Latihan B

```
1  import cv2 #import module opencv
2  import mediapipe as mp
3
4  capture = cv2.VideoCapture(0)
5
6  mediapipehand = mp.solutions.hands
7  hand = mediapipehand.Hands(max_num_hands=1)
8  mpdraw = mp.solutions.drawing_utils
9
10 while True:
11     success, frame = capture.read()
12     if not success:
13         break
14
15     frame = cv2.flip(frame, 1)
16
17     imgRGB = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
18     results = hand.process(imgRGB)
19
20     if results.multi_hand_landmarks:
21         for titiktangan in results.multi_hand_landmarks:
22
23             mpdraw.draw_landmarks(frame, titiktangan, mediapipehand.HAND_CONNECTIONS)
24
25             #Mendefinisikan Titik tangan
26             pergelangan = titiktangan.landmark[0]
27             ujungjempol = titiktangan.landmark[4]
28             ujungkelingking = titiktangan.landmark[20]
29
30             h, w, c = frame.shape
```

```
31
32     #koordinat pixel
33     jempol = int(ujungjempol.x * w)
34     kelingking = int(ujungkelingking.x * w)
35
36     #cek posisi tangan (depan atau belakang)
37     if jempol > kelingking:
38         posisi = "Tangan Belakang"
39     else:
40         posisi = "Telapak Tangan (depan)"
41
42     cv2.putText(frame, posisi, (10, 50), cv2.FONT_HERSHEY_PLAIN, 1, (0, 0, 255), 2)
43
44     cv2.imshow("Frame", frame)
45     if cv2.waitKey(1) & 0xFF == ord('q'):
46         break
47     capture.release()
48     cv2.destroyAllWindows()
```

Program ini merupakan pengembangan lanjutan dari sistem deteksi tangan sebelumnya, dengan penambahan fitur identifikasi posisi tangan berdasarkan orientasi telapak atau punggung tangan. Pada tahap awal dilakukan proses impor library OpenCV dan MediaPipe, serta inisialisasi kamera menggunakan `cv2.VideoCapture(0)` sebagai sumber input citra secara real-time. Proses ini sama dengan program sebelumnya, yaitu memanfaatkan webcam untuk menangkap frame video secara berkelanjutan.

Modul `mp.solutions.hands` diinisialisasi dengan parameter `max_num_hands=1`, yang membatasi sistem hanya mendeteksi satu tangan untuk meningkatkan efisiensi pemrosesan. Variabel `mpdraw` digunakan untuk menggambar landmark dan koneksi antar titik tangan pada frame.

Berbeda dengan program sebelumnya, pada program ini ditambahkan perintah `cv2.flip(frame, 1)` yang berfungsi untuk membalik tampilan frame secara horizontal (mirror). Proses ini bertujuan agar tampilan kamera menyerupai cermin sehingga gerakan tangan pengguna terlihat lebih natural dan sesuai dengan persepsi visual.

Setelah frame diperoleh, dilakukan konversi warna dari format BGR ke RGB sebelum diproses oleh model MediaPipe. Jika sistem mendeteksi tangan melalui atribut `results.multi_hand_landmarks`, maka landmark tangan akan digambar pada frame menggunakan fungsi `draw_landmarks()`.

Program kemudian mendefinisikan beberapa titik penting pada tangan, yaitu landmark indeks 0 sebagai pergelangan tangan, indeks 4 sebagai ujung jempol, dan indeks 20 sebagai ujung kelingking. Landmark-landmark tersebut digunakan sebagai referensi untuk menentukan orientasi tangan. Untuk memperoleh posisi dalam bentuk koordinat piksel, nilai koordinat ternormalisasi dikalikan dengan lebar frame (w) yang diperoleh dari `frame.shape`.

Logika penentuan posisi tangan dilakukan dengan membandingkan posisi horizontal (sumbu x) antara ujung jempol dan ujung kelingking. Apabila koordinat x jempol lebih besar daripada koordinat x kelingking, maka sistem mengidentifikasi posisi sebagai “Tangan Belakang”. Sebaliknya, jika tidak memenuhi kondisi tersebut, maka sistem menampilkan “Telapak Tangan (depan)”. Hasil klasifikasi ini kemudian ditampilkan pada layar menggunakan fungsi `cv2.putText()`.

Secara keseluruhan, program ini tidak hanya mendeteksi dan menampilkan landmark tangan secara real-time, tetapi juga melakukan analisis sederhana terhadap orientasi tangan berdasarkan perbandingan posisi dua titik ekstrem jari. Pendekatan ini menunjukkan bahwa data landmark dari MediaPipe dapat dimanfaatkan lebih lanjut untuk melakukan interpretasi posisi dan arah tangan, sehingga sistem dapat dikembangkan menjadi aplikasi pengenalan gesture atau interaksi berbasis gerakan tangan yang lebih kompleks.

6. Kesimpulan

Setelah dilakukan percobaan atau praktikum dapat disimpulkan bahwa :

1. Library OpenCV dan MediaPipe dapat diimplementasikan secara efektif untuk membangun sistem deteksi dan analisis tangan secara real-time menggunakan webcam.
2. Sistem mampu membaca citra dari kamera, mengonversi format warna sesuai kebutuhan model, dan memproses citra untuk mendeteksi keberadaan tangan.
3. Program berhasil mendeteksi ada atau tidaknya tangan pada frame video.
4. Sistem mampu menampilkan 21 titik landmark tangan beserta garis penghubungnya sehingga struktur tangan dapat divisualisasikan dengan jelas.
5. Data koordinat setiap landmark berhasil diekstraksi dalam bentuk nilai ternormalisasi yang dapat digunakan untuk analisis lanjutan, seperti pengenalan gesture.
6. Sistem dapat mengklasifikasikan tangan sebagai tangan kanan atau tangan kiri berdasarkan fitur handedness dari MediaPipe.
7. Program mampu mengidentifikasi orientasi tangan (telapak atau punggung tangan) berdasarkan perbandingan posisi koordinat ujung jempol dan ujung kelingking.
8. Secara keseluruhan, sistem yang dikembangkan dapat menjadi dasar pengembangan aplikasi interaksi manusia dan komputer berbasis gesture yang lebih kompleks di masa mendatang.

7. Saran

Berdasarkan hasil praktikum yang telah dilakukan, terdapat beberapa saran untuk pengembangan sistem selanjutnya :

1. sistem dapat ditingkatkan dengan menambahkan metode pengenalan gesture tertentu, seperti gerakan membuka dan menutup tangan atau kombinasi posisi jari, sehingga aplikasi menjadi lebih interaktif.
2. akurasi deteksi dapat ditingkatkan dengan melakukan pengaturan parameter seperti nilai confidence atau dengan menambahkan pencahayaan yang lebih stabil agar hasil deteksi lebih optimal.
3. sistem dapat dikembangkan untuk diintegrasikan dengan perangkat lain, seperti mikrokontroler atau aplikasi berbasis IoT, sehingga gerakan tangan dapat digunakan sebagai media kontrol suatu perangkat.

8. Referensi

- Diógenes Do Rego, I., & De Sousa, V. A. (2021). Solution for Interference in Hotspot Scenarios Applying Q-Learning on FFR-Based ICIC Techniques. *Sensors*, 21(23), 7899. <https://doi.org/10.3390/s21237899>
- Lugaresi, C., Tang, J., Nash, H., McClanahan, C., Uboweja, E., Hays, M., Zhang, F., Chang, C.-L., Yong, M. G., Lee, J., Chang, W.-T., Hua, W., Georg, M., & Grundmann, M. (t.t.). *MediaPipe: A Framework for Building Perception Pipelines*.
- Zhang, F., Bazarevsky, V., Vakunov, A., Tkachenka, A., Sung, G., Chang, C.-L., & Grundmann, M. (2020). *MediaPipe Hands: On-device Real-time Hand Tracking* (arXiv:2006.10214). arXiv. <https://doi.org/10.48550/arXiv.2006.10214>